Oui — voici un **aide-mémoire COBOL pour débutants**, pensé comme une **fiche de base complète**.

Aide-mémoire COBOL — débutant

1. À quoi sert COBOL ?

COBOL = **COmmon Business-Oriented Language**.

Il est surtout utilisé pour :

- la gestion,
- la comptabilité,
- la banque,
- les assurances,
- les traitements de fichiers,
- les applications métier anciennes mais encore très utilisées.

COBOL est un langage :

- très lisible,
- très verbeux,
- orienté traitement de données.

---

2. Structure d’un programme COBOL

Un programme COBOL est souvent découpé en **4 divisions** :

IDENTIFICATION DIVISION.
ENVIRONMENT DIVISION.
DATA DIVISION.
PROCEDURE DIVISION.

Rôle de chaque division

IDENTIFICATION DIVISION

Contient l’identité du programme.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BONJOUR.
```

ENVIRONMENT DIVISION

Décrit l’environnement, surtout pour les fichiers.

```
ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
```

DATA DIVISION

Déclare les variables et les fichiers.

```
DATA DIVISION.
WORKING-STORAGE SECTION.
```

PROCEDURE DIVISION

Contient les instructions exécutées.

```
PROCEDURE DIVISION.
   DISPLAY "Bonjour".
   STOP RUN.
```

---

3. Programme minimal

```
IDENTIFICATION DIVISION.
PROGRAM-ID. HELLO.

PROCEDURE DIVISION.
   DISPLAY "Bonjour le monde".
   STOP RUN.
```

---

4. Règles d’écriture de base

Les noms

Les noms de variables sont souvent écrits avec des tirets :

NOM-CLIENT
MONTANT-TOTAL
COMPTEUR

Le point

En COBOL, le . termine souvent une phrase/instruction importante.

Majuscules

Traditionnellement COBOL s'écrit en majuscules, mais ce n'est pas toujours obligatoire selon l'outil.

Commentaires

Selon le compilateur, on peut voir :

*> Ceci est un commentaire

---

5. Les sections de données

La plus utilisée pour débuter :

WORKING-STORAGE SECTION

Pour les variables normales du programme.

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOM        PIC A(20).
01 WS-AGE        PIC 9(3).
01 WS-SALAIRE    PIC 9(5)V99.
```

---

6. Déclaration des variables

Niveaux hiérarchiques

Les variables sont déclarées avec des **numéros de niveau** :

- `01` : variable principale
- `05`, `10`, etc. : sous-zones
- `77` : ancienne façon pour variable simple
- `88` : condition nommée

Exemple :

```
01 WS-PERSONNE.
   05 WS-NOM     PIC A(20).
   05 WS-AGE     PIC 9(3).
```

---

7. PIC : le format des données

`PIC` = **Picture**, décrit le format.

Types courants

Alphanumérique

PIC X(10)

Contient n'importe quel caractère.

Alphabétique

PIC A(10)

Contient des lettres.

Numérique

PIC 9(5)

Contient 5 chiffres.

Décimal implicite

PIC 9(5)V99

Exemple : `12345V67` = 12345,67

Le `V` n'est **pas affiché**, il représente la position de la virgule décimale.

Signe

PIC S9(5)V99

---

8. VALUE : valeur initiale

```
01 WS-NOM       PIC X(20) VALUE "INCONNU".
01 WS-AGE       PIC 9(3)  VALUE 0.
01 WS-OK        PIC X     VALUE "N".
```

---

9. Variables structurées

```
01 WS-DATE.
   05 WS-ANNEE   PIC 9(4).
   05 WS-MOIS    PIC 9(2).
   05 WS-JOUR    PIC 9(2).
```

On peut aussi déplacer toute la structure d'un coup si les formats sont compatibles.

---

10. Affichage et saisie

DISPLAY

Affiche à l'écran.

```
DISPLAY "Bonjour".
DISPLAY WS-NOM.
DISPLAY "Age : " WS-AGE.
```

ACCEPT

Lit une valeur entrée par l’utilisateur.

```
ACCEPT WS-NOM.
ACCEPT WS-AGE.
```

Exemple :

```
DISPLAY "Entrez votre nom : ".
ACCEPT WS-NOM.
DISPLAY "Bonjour " WS-NOM.
```

---

11. Affectation de valeurs

MOVE

Copie une valeur.

```
MOVE "Alice" TO WS-NOM.
MOVE 25 TO WS-AGE.
MOVE WS-AGE TO WS-AGE-SAUVE.
```

---

12. Calculs de base

ADD

```
ADD 1 TO WS-COMPTEUR.
ADD WS-A TO WS-B.
ADD WS-A WS-B TO WS-TOTAL.
```

SUBTRACT

```
SUBTRACT 1 FROM WS-COMPTEUR.
SUBTRACT WS-REM TO WS-TOTAL.
```

MULTIPLY

```
MULTIPLY WS-PRIX BY WS-QTE.
```

MULTIPLY WS-PRIX BY WS-QTE GIVING WS-TOTAL.

DIVIDE

DIVIDE WS-TOTAL BY WS-NB.
DIVIDE WS-TOTAL BY WS-NB GIVING WS-MOYENNE.

COMPUTE

Souvent plus pratique.

COMPUTE WS-TOTAL = WS-PRIX * WS-QTE.
COMPUTE WS-MOYENNE = WS-SOMME / WS-NB.
COMPUTE WS-TTC = WS-HT * 1.20.

---

13. Conditions

IF / ELSE / END-IF

```
IF WS-AGE >= 18
   DISPLAY "Majeur"
ELSE
   DISPLAY "Mineur"
END-IF.
```

Opérateurs relationnels

- `=`
- `>`
- `<`
- `>=`
- `<=`
- `NOT =`

Exemple :

```
IF WS-NOTE > 10
   DISPLAY "Admis"
END-IF.
```

Conditions combinées

```
IF WS-AGE >= 18 AND WS-AGE <= 65
   DISPLAY "Age actif"
END-IF.

IF WS-PAYS = "FR" OR WS-PAYS = "BE"
   DISPLAY "Pays autorise"
END-IF.
```

---

14. EVALUATE = sorte de CASE

Très utile pour remplacer plusieurs `IF`.

```
EVALUATE WS-NOTE
   WHEN 20
      DISPLAY "Parfait"
   WHEN 10 THRU 19
      DISPLAY "OK"
   WHEN OTHER
      DISPLAY "Insuffisant"
END-EVALUATE.
```

On peut aussi faire :

```
EVALUATE TRUE
   WHEN WS-AGE < 18
      DISPLAY "Mineur"
   WHEN WS-AGE >= 18
      DISPLAY "Majeur"
END-EVALUATE.
```

---

15. Boucles et enchaînement

PERFORM

Instruction de base pour exécuter un bloc ou un paragraphe.

Exécuter un paragraphe

```
PERFORM AFFICHER-MESSAGE.
```

Exemple avec paragraphe

```
PROCEDURE DIVISION.
   PERFORM AFFICHER-MESSAGE
   STOP RUN.

AFFICHER-MESSAGE.
   DISPLAY "Bonjour".
```

PERFORM UNTIL

```
PERFORM UNTIL WS-COMPTEUR > 5
   DISPLAY WS-COMPTEUR
   ADD 1 TO WS-COMPTEUR
END-PERFORM.
```

PERFORM VARYING

Très proche d'une boucle `for`.

```
PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 10
   DISPLAY WS-I
END-PERFORM.
```

---

16. Les paragraphes

Le code peut être organisé en paragraphes :

```
PROCEDURE DIVISION.
   PERFORM DEBUT-TRAITEMENT
   PERFORM FIN-TRAITEMENT
   STOP RUN.

DEBUT-TRAITEMENT.
   DISPLAY "Debut".

FIN-TRAITEMENT.
   DISPLAY "Fin".
```

---

17. Conditions nommées avec le niveau 88

Très pratique en COBOL.

```
01 WS-STATUT      PIC X.
   88 STATUT-OK   VALUE "O".
   88 STATUT-KO   VALUE "N".
```

Utilisation :

```
MOVE "O" TO WS-STATUT.

IF STATUT-OK
    DISPLAY "Succes"
END-IF.
```

On peut aussi faire :

```
SET STATUT-OK TO TRUE.
```

---

18. Chaînes de caractères

STRING

Concatène des chaînes.

```
STRING WS-PRENOM DELIMITED BY SPACE
       " "
       WS-NOM DELIMITED BY SPACE
INTO WS-NOM-COMPLET.
```

UNSTRING

Découpe une chaîne.

```
UNSTRING WS-LIGNE
   DELIMITED BY ";"
   INTO WS-CHAMP1
       WS-CHAMP2
       WS-CHAMP3.
```

INSPECT

Compte ou remplace des caractères.

```
INSPECT WS-TEXTE TALLYING WS-NB FOR ALL "A".
```

---

19. Tableaux

En COBOL, un tableau se fait avec `OCCURS`.

```
01 WS-TABLEAU.
  05 WS-VALEUR PIC 9(3) OCCURS 5 TIMES.
```

Accès :

```
MOVE 100 TO WS-VALEUR(1).
DISPLAY WS-VALEUR(1).
```

Avec index ou compteur :

```
01 WS-I PIC 9(2).

PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
   DISPLAY WS-VALEUR(WS-I)
END-PERFORM.
```

---

20. Fichiers : notions de base

COBOL est très utilisé pour les fichiers.

Déclaration d’un fichier

ENVIRONMENT DIVISION

```
ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
   SELECT FICHIER-CLIENT ASSIGN TO "clients.txt"
      ORGANIZATION IS LINE SEQUENTIAL.
```

DATA DIVISION

```
DATA DIVISION.
FILE SECTION.
FD FICHIER-CLIENT.
01 ENR-CLIENT      PIC X(80).
```

---

21. Lecture de fichier

Variables utiles :

```
WORKING-STORAGE SECTION.
01 WS-FIN         PIC X VALUE "N".
  88 FIN-FICHIER  VALUE "O".
```

Code :

```
PROCEDURE DIVISION.
   OPEN INPUT FICHIER-CLIENT

   PERFORM UNTIL FIN-FICHIER
      READ FICHIER-CLIENT
         AT END
            MOVE "O" TO WS-FIN
         NOT AT END
            DISPLAY ENR-CLIENT
      END-READ
   END-PERFORM

   CLOSE FICHIER-CLIENT
   STOP RUN.
```

---

22. Écriture de fichier

OPEN OUTPUT FICHIER-CLIENT

MOVE "LIGNE 1" TO ENR-CLIENT
WRITE ENR-CLIENT

CLOSE FICHIER-CLIENT.

---

23. Verbes fichiers essentiels

- `OPEN INPUT fichier`
- `OPEN OUTPUT fichier`
- `OPEN I-O fichier`
- `READ fichier`
- `WRITE enregistrement`
- `REWRITE enregistrement`
- `CLOSE fichier`

---

24. Formatage d’affichage

On peut mettre des symboles d’édition dans les `PIC`.

Exemples :

01 WS-MONTANT-BRUT PIC 9(5)V99 VALUE 12345.67.
01 WS-MONTANT-AFF PIC ZZZ.ZZ9,99.

Le `Z` masque les zéros à gauche.

---

25. Variables spéciales utiles

Compteur

01 WS-COMPTEUR PIC 9(4) VALUE 0.

Booléen simulé

COBOL n’a pas toujours de vrai booléen classique, on utilise souvent :

01 WS-FLAG PIC X VALUE "N".

ou mieux avec 88.

---

26. Exemples d’instructions très fréquentes

Initialiser

INITIALIZE WS-PERSONNE.

Met les champs à zéro ou espace selon le type.

Continuer sans rien faire

CONTINUE.

Arrêter le programme

STOP RUN.

Sortir d’un paragraphe

EXIT.

---

27. Exemple complet simple : calcul de moyenne

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MOYENNE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-N1        PIC 9(3).
01 WS-N2        PIC 9(3).
01 WS-MOYENNE   PIC 9(3)V99.

PROCEDURE DIVISION.
   DISPLAY "Entrez note 1 : "
   ACCEPT WS-N1

   DISPLAY "Entrez note 2 : "
   ACCEPT WS-N2

   COMPUTE WS-MOYENNE = (WS-N1 + WS-N2) / 2

   DISPLAY "Moyenne = " WS-MOYENNE

   STOP RUN.
```

---

28. Exemple complet simple : boucle

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BOUCLE1.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I PIC 9(2) VALUE 1.

PROCEDURE DIVISION.
   PERFORM UNTIL WS-I > 5
      DISPLAY "Valeur = " WS-I
      ADD 1 TO WS-I
   END-PERFORM

   STOP RUN.
```

---

29. Exemple complet simple : IF

```
IDENTIFICATION DIVISION.
PROGRAM-ID. TESTAGE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-AGE PIC 9(3).

PROCEDURE DIVISION.
   DISPLAY "Entrez votre age : "
   ACCEPT WS-AGE

   IF WS-AGE >= 18
      DISPLAY "Vous etes majeur"
   ELSE
      DISPLAY "Vous etes mineur"
   END-IF

   STOP RUN.
```

---

30. Erreurs fréquentes chez les débutants

Oublier `END-IF`

```
IF A = B
   DISPLAY "OK"
END-IF
```

Mélanger les types

Mettre du texte dans une variable numérique provoque souvent une erreur.

Oublier d'initialiser une variable

Toujours penser à `VALUE` ou `INITIALIZE`.

Mauvais format `PIC`

Exemple : stocker `12345` dans `PIC 9(3)` ne rentre pas.

Oublier `OPEN` avant `READ`

Il faut ouvrir le fichier avant lecture/écriture.

Oublier `STOP RUN`

Le programme doit se terminer proprement.

---

31. Mots-clés très utiles à retenir

Voici le noyau à connaître :

IDENTIFICATION DIVISION
ENVIRONMENT DIVISION
DATA DIVISION
PROCEDURE DIVISION
WORKING-STORAGE SECTION
FILE SECTION
PIC
VALUE
MOVE
DISPLAY
ACCEPT
ADD
SUBTRACT
MULTIPLY
DIVIDE
COMPUTE
IF
ELSE
END-IF
EVALUATE
WHEN
END-EVALUATE
PERFORM
UNTIL
VARYING
OPEN
READ
WRITE
CLOSE
STOP RUN

---

32. Ordre classique pour écrire un petit programme

1. Déclarer le `PROGRAM-ID`
2. Déclarer les variables dans `WORKING-STORAGE`
3. Écrire les traitements dans `PROCEDURE DIVISION`
4. Afficher ou lire les données avec `DISPLAY` / `ACCEPT`
5. Faire les calculs avec `COMPUTE` ou `ADD`
6. Terminer avec `STOP RUN`

---

33. Gabarit de départ pratique

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MONPROG.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOM       PIC X(20) VALUE SPACES.
01 WS-AGE       PIC 9(3)  VALUE 0.

PROCEDURE DIVISION.
   DISPLAY "Nom ? "
   ACCEPT WS-NOM

   DISPLAY "Age ? "
   ACCEPT WS-AGE

   IF WS-AGE >= 18
      DISPLAY WS-NOM " est majeur"
   ELSE
      DISPLAY WS-NOM " est mineur"
   END-IF

   STOP RUN.
```

---

34. Version ultra-résumée à retenir par cœur

Déclarer

```
01 WS-NOM PIC X(20).
01 WS-AGE PIC 9(3).
```

Afficher

```
DISPLAY WS-NOM.
```

Lire

```
ACCEPT WS-NOM.
```

Copier

```
MOVE 10 TO WS-AGE.
```

Calculer

```
COMPUTE WS-TOTAL = WS-A + WS-B.
```

Tester

```
IF WS-AGE >= 18
   DISPLAY "OK"
END-IF.
```

Boucler

```
PERFORM UNTIL WS-I > 10
   ADD 1 TO WS-I
END-PERFORM.
```

---

35. Ce qu’un débutant doit apprendre en premier

Commence par maîtriser dans cet ordre :

1. la structure du programme,
2. les variables et `PIC`,
3. `DISPLAY`, `ACCEPT`, `MOVE`,
4. `IF` et `EVALUATE`,
5. `COMPUTE`,
6. `PERFORM`,
7. la lecture/écriture de fichiers.

---

36. Mini fiche de traduction mentale

Quand tu lis du COBOL, pense :

- `MOVE` = affecter
- `DISPLAY` = afficher
- `ACCEPT` = saisir
- `COMPUTE` = calculer
- `IF` = si
- `EVALUATE` = selon / case
- `PERFORM` = exécuter / boucler
- `READ` = lire
- `WRITE` = écrire
- `STOP RUN` = fin du programme

Je peux aussi te faire une **version encore plus propre en mode PDF/fiche de révision**, ou une **version avec exercices + corrigés**.

1. Bubble sort sur un tableau

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BUBBLE-SORT.
```

```
DATA DIVISION.
WORKING-STORAGE SECTION.

*> Indices de boucle
01 WS-I              PIC 9(2).
01 WS-J              PIC 9(2).

*> Variable temporaire pour échanger 2 valeurs
01 WS-TMP            PIC 9(3).

*> Tableau de 5 entiers
01 WS-TAB.
  05 WS-VAL          PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   *> Initialisation du tableau
   MOVE 45 TO WS-VAL(1)
   MOVE 12 TO WS-VAL(2)
   MOVE 89 TO WS-VAL(3)
   MOVE 03 TO WS-VAL(4)
   MOVE 27 TO WS-VAL(5)

   DISPLAY "Avant tri :"
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
      DISPLAY WS-VAL(WS-I)
   END-PERFORM

   *> Bubble sort
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 4
      PERFORM VARYING WS-J FROM 1 BY 1 UNTIL WS-J > 5 - WS-I
         IF WS-VAL(WS-J) > WS-VAL(WS-J + 1)
            MOVE WS-VAL(WS-J)     TO WS-TMP
            MOVE WS-VAL(WS-J + 1) TO WS-VAL(WS-J)
            MOVE WS-TMP           TO WS-VAL(WS-J + 1)
         END-IF
      END-PERFORM
   END-PERFORM

   DISPLAY "Apres tri :"
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
      DISPLAY WS-VAL(WS-I)
   END-PERFORM

   STOP RUN.
```

2. Recherche linéaire dans un tableau

```
IDENTIFICATION DIVISION.
PROGRAM-ID. RECHERCHE-LINEAIRE.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-I              PIC 9(2).
01 WS-CIBLE          PIC 9(3) VALUE 27.
01 WS-TROUVE         PIC X VALUE "N".
  88 TROUVE         VALUE "O".
  88 NON-TROUVE     VALUE "N".
01 WS-POSITION       PIC 9(2) VALUE 0.

01 WS-TAB.
  05 WS-VAL         PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   MOVE 10 TO WS-VAL(1)
   MOVE 27 TO WS-VAL(2)
   MOVE 33 TO WS-VAL(3)
   MOVE 41 TO WS-VAL(4)
   MOVE 50 TO WS-VAL(5)

   *> Parcours du tableau jusqu'a trouver la valeur
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5 OR TROUVE
      IF WS-VAL(WS-I) = WS-CIBLE
         SET TROUVE TO TRUE
         MOVE WS-I TO WS-POSITION
      END-IF
   END-PERFORM

   IF TROUVE
      DISPLAY "Valeur trouvee a la position : " WS-POSITION
   ELSE
      DISPLAY "Valeur non trouvee"
   END-IF

   STOP RUN.
```

---

3. Recherche binaire

À utiliser sur un **tableau déjà trié**.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. RECHERCHE-BINAIRE.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-LOW            PIC 9(2) VALUE 1.
01 WS-HIGH           PIC 9(2) VALUE 5.
01 WS-MID            PIC 9(2).
01 WS-CIBLE          PIC 9(3) VALUE 33.
01 WS-POSITION       PIC 9(2) VALUE 0.
01 WS-TROUVE         PIC X VALUE "N".
  88 TROUVE          VALUE "O".

01 WS-TAB.
  05 WS-VAL          PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   *> Tableau trie
   MOVE 10 TO WS-VAL(1)
   MOVE 20 TO WS-VAL(2)
   MOVE 33 TO WS-VAL(3)
   MOVE 40 TO WS-VAL(4)
   MOVE 55 TO WS-VAL(5)

   PERFORM UNTIL WS-LOW > WS-HIGH OR TROUVE
      COMPUTE WS-MID = (WS-LOW + WS-HIGH) / 2

      IF WS-VAL(WS-MID) = WS-CIBLE
         SET TROUVE TO TRUE
         MOVE WS-MID TO WS-POSITION
      ELSE
         IF WS-CIBLE < WS-VAL(WS-MID)
            SUBTRACT 1 FROM WS-MID GIVING WS-HIGH
         ELSE
            ADD 1 TO WS-MID GIVING WS-LOW
         END-IF
      END-IF
   END-PERFORM

   IF TROUVE
      DISPLAY "Trouve a la position : " WS-POSITION
   ELSE
      DISPLAY "Non trouve"
   END-IF

   STOP RUN.
```

4. Trouver le max, le min, la somme et la moyenne

```
IDENTIFICATION DIVISION.
PROGRAM-ID. STATS-TABLEAU.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-I              PIC 9(2).
01 WS-SOMME           PIC 9(5) VALUE 0.
01 WS-MAX             PIC 9(3).
01 WS-MIN             PIC 9(3).
01 WS-MOYENNE         PIC 9(3)V99.

01 WS-TAB.
  05 WS-VAL          PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   MOVE 12 TO WS-VAL(1)
   MOVE 35 TO WS-VAL(2)
   MOVE 08 TO WS-VAL(3)
   MOVE 47 TO WS-VAL(4)
   MOVE 19 TO WS-VAL(5)

   *> On initialise max et min avec la premiere valeur
   MOVE WS-VAL(1) TO WS-MAX
   MOVE WS-VAL(1) TO WS-MIN

   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
      ADD WS-VAL(WS-I) TO WS-SOMME

      IF WS-VAL(WS-I) > WS-MAX
         MOVE WS-VAL(WS-I) TO WS-MAX
      END-IF

      IF WS-VAL(WS-I) < WS-MIN
         MOVE WS-VAL(WS-I) TO WS-MIN
      END-IF
   END-PERFORM

   COMPUTE WS-MOYENNE = WS-SOMME / 5

   DISPLAY "Somme    : " WS-SOMME
   DISPLAY "Maximum  : " WS-MAX
   DISPLAY "Minimum  : " WS-MIN
```

```
  DISPLAY "Moyenne  : " WS-MOYENNE

  STOP RUN.
```

---

5. Factorielle

```
IDENTIFICATION DIVISION.
PROGRAM-ID. FACTORIELLE.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-N              PIC 9(2) VALUE 5.
01 WS-I              PIC 9(2).
01 WS-RESULTAT       PIC 9(9) VALUE 1.

PROCEDURE DIVISION.
  *> 5! = 5 * 4 * 3 * 2 * 1
  IF WS-N = 0
     MOVE 1 TO WS-RESULTAT
  ELSE
     PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-N
        MULTIPLY WS-I BY WS-RESULTAT
     END-PERFORM
  END-IF

  DISPLAY "Factorielle = " WS-RESULTAT
  STOP RUN.
```

---

6. Fibonacci

```
IDENTIFICATION DIVISION.
PROGRAM-ID. FIBONACCI.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-N              PIC 9(2) VALUE 10.
01 WS-I              PIC 9(2).
01 WS-A              PIC 9(5) VALUE 0.
01 WS-B              PIC 9(5) VALUE 1.
01 WS-C              PIC 9(5) VALUE 0.
```

```
PROCEDURE DIVISION.
  DISPLAY "Suite de Fibonacci : "
  DISPLAY WS-A
  DISPLAY WS-B

  PERFORM VARYING WS-I FROM 3 BY 1 UNTIL WS-I > WS-N
     COMPUTE WS-C = WS-A + WS-B
     DISPLAY WS-C
     MOVE WS-B TO WS-A
     MOVE WS-C TO WS-B
  END-PERFORM

  STOP RUN.
```

---

7. Vérifier si un nombre est premier

```
IDENTIFICATION DIVISION.
PROGRAM-ID. NOMBRE-PREMIER.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-N              PIC 9(3) VALUE 29.
01 WS-I              PIC 9(3).
01 WS-RESTE           PIC 9(3).
01 WS-PREMIER         PIC X VALUE "O".
  88 EST-PREMIER     VALUE "O".
  88 PAS-PREMIER     VALUE "N".

PROCEDURE DIVISION.
  IF WS-N < 2
     SET PAS-PREMIER TO TRUE
  ELSE
     PERFORM VARYING WS-I FROM 2 BY 1 UNTIL WS-I >= WS-N OR PAS-PREMIER
        DIVIDE WS-N BY WS-I GIVING WS-RESTE REMAINDER WS-RESTE
        IF WS-RESTE = 0
           SET PAS-PREMIER TO TRUE
        END-IF
     END-PERFORM
  END-IF

  IF EST-PREMIER
     DISPLAY WS-N " est premier"
  ELSE
     DISPLAY WS-N " n'est pas premier"
```

```
  END-IF

  STOP RUN.
```

---

8. Inverser une chaîne

```
IDENTIFICATION DIVISION.
PROGRAM-ID. INVERSER-CHAINE.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-TEXTE          PIC X(20) VALUE "COBOL".
01 WS-INVERSE        PIC X(20) VALUE SPACES.
01 WS-LEN            PIC 9(2).
01 WS-I              PIC 9(2).

PROCEDURE DIVISION.
  *> On calcule la longueur logique ici a la main
  MOVE 5 TO WS-LEN

  PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-LEN
     MOVE WS-TEXTE(WS-LEN - WS-I + 1:1) TO WS-INVERSE(WS-I:1)
  END-PERFORM

  DISPLAY "Texte   : " WS-TEXTE
  DISPLAY "Inverse : " WS-INVERSE

  STOP RUN.
```

---

9. Vérifier si un mot est un palindrome

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PALINDROME.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-TEXTE          PIC X(20) VALUE "RADAR".
01 WS-I              PIC 9(2).
01 WS-LEN            PIC 9(2) VALUE 5.
01 WS-OK             PIC X VALUE "O".
  88 EST-PALINDROME   VALUE "O".
```

```
  88 PAS-PALINDROME   VALUE "N".

PROCEDURE DIVISION.
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-LEN / 2 OR
PAS-PALINDROME
      IF WS-TEXTE(WS-I:1) NOT = WS-TEXTE(WS-LEN - WS-I + 1:1)
         SET PAS-PALINDROME TO TRUE
      END-IF
   END-PERFORM

   IF EST-PALINDROME
      DISPLAY "C'est un palindrome"
   ELSE
      DISPLAY "Ce n'est pas un palindrome"
   END-IF

   STOP RUN.
```

---

10. Compter les voyelles dans une chaîne

```
IDENTIFICATION DIVISION.
PROGRAM-ID. COMPTER-VOYELLES.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-TEXTE           PIC X(20) VALUE "BONJOUR COBOL".
01 WS-I              PIC 9(2).
01 WS-CAR            PIC X.
01 WS-NB-VOYELLES     PIC 9(2) VALUE 0.
01 WS-LEN            PIC 9(2) VALUE 13.

PROCEDURE DIVISION.
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-LEN
      MOVE WS-TEXTE(WS-I:1) TO WS-CAR

      EVALUATE WS-CAR
         WHEN "A"
         WHEN "E"
         WHEN "I"
         WHEN "O"
         WHEN "U"
         WHEN "Y"
```

```
            ADD 1 TO WS-NB-VOYELLES
         WHEN OTHER
            CONTINUE
      END-EVALUATE
   END-PERFORM

   DISPLAY "Nombre de voyelles : " WS-NB-VOYELLES
   STOP RUN.
```

---

11. Vérifier si un tableau est trié

```
IDENTIFICATION DIVISION.
PROGRAM-ID. TABLEAU-TRIE.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-I              PIC 9(2).
01 WS-TRIE            PIC X VALUE "O".
  88 EST-TRIE         VALUE "O".
  88 PAS-TRIE         VALUE "N".

01 WS-TAB.
  05 WS-VAL           PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   MOVE 10 TO WS-VAL(1)
   MOVE 20 TO WS-VAL(2)
   MOVE 30 TO WS-VAL(3)
   MOVE 40 TO WS-VAL(4)
   MOVE 50 TO WS-VAL(5)

   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 4 OR PAS-TRIE
      IF WS-VAL(WS-I) > WS-VAL(WS-I + 1)
         SET PAS-TRIE TO TRUE
      END-IF
   END-PERFORM

   IF EST-TRIE
      DISPLAY "Le tableau est trie"
   ELSE
      DISPLAY "Le tableau n'est pas trie"
   END-IF

   STOP RUN.
```

12. Fusion de 2 tableaux triés

Très classique dans les questions type algo.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. FUSION-TABLEAUX.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 I                PIC 9(2) VALUE 1.
01 J                PIC 9(2) VALUE 1.
01 K                PIC 9(2) VALUE 1.

01 TAB1.
  05 T1             PIC 9(3) OCCURS 3 TIMES.
01 TAB2.
  05 T2             PIC 9(3) OCCURS 3 TIMES.
01 TAB3.
  05 T3             PIC 9(3) OCCURS 6 TIMES.

PROCEDURE DIVISION.
   MOVE 10 TO T1(1)
   MOVE 30 TO T1(2)
   MOVE 50 TO T1(3)

   MOVE 20 TO T2(1)
   MOVE 40 TO T2(2)
   MOVE 60 TO T2(3)

   *> Fusion tant que les 2 tableaux ont encore des valeurs
   PERFORM UNTIL I > 3 OR J > 3
      IF T1(I) <= T2(J)
         MOVE T1(I) TO T3(K)
         ADD 1 TO I
      ELSE
         MOVE T2(J) TO T3(K)
         ADD 1 TO J
      END-IF
      ADD 1 TO K
   END-PERFORM

   *> Copier le reste de TAB1
   PERFORM UNTIL I > 3
```

```
    MOVE T1(I) TO T3(K)
    ADD 1 TO I
    ADD 1 TO K
  END-PERFORM

  *> Copier le reste de TAB2
  PERFORM UNTIL J > 3
    MOVE T2(J) TO T3(K)
    ADD 1 TO J
    ADD 1 TO K
  END-PERFORM

  DISPLAY "Tableau fusionne :"
  PERFORM VARYING K FROM 1 BY 1 UNTIL K > 6
    DISPLAY T3(K)
  END-PERFORM

  STOP RUN.
```

---

13. Parcours de graphe simple en BFS

COBOL n’est pas le plus pratique pour les graphes, mais ça peut tomber en test.
Ici on utilise :

- une **matrice d’adjacence**,
- un tableau `VISITED`,
- une petite **file**.

Graphe :

- 1 relié à 2 et 3
- 2 relié à 4
- 3 relié à 4

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BFS-GRAPHE.

DATA DIVISION.
WORKING-STORAGE SECTION.
```

```
01 WS-N              PIC 9 VALUE 4.
01 WS-I              PIC 9.
01 WS-J              PIC 9.
01 WS-CURRENT        PIC 9.

*> File pour BFS
01 WS-FRONT          PIC 9 VALUE 1.
01 WS-REAR           PIC 9 VALUE 1.
01 WS-QUEUE.
  05 WS-Q            PIC 9 OCCURS 10 TIMES VALUE 0.

*> Tableau des sommets visites
01 WS-VISITED.
  05 WS-VIS          PIC 9 OCCURS 4 TIMES VALUE 0.

*> Matrice d'adjacence 4x4
01 WS-GRAPH.
  05 WS-ROW OCCURS 4 TIMES.
    10 WS-COL        PIC 9 OCCURS 4 TIMES VALUE 0.

PROCEDURE DIVISION.
  *> Initialisation du graphe
  *> 1 -> 2, 3
  MOVE 1 TO WS-COL(1,2)
  MOVE 1 TO WS-COL(1,3)

  *> 2 -> 4
  MOVE 1 TO WS-COL(2,4)

  *> 3 -> 4
  MOVE 1 TO WS-COL(3,4)

  *> On commence par le sommet 1
  MOVE 1 TO WS-Q(WS-REAR)
  MOVE 1 TO WS-VIS(1)

  DISPLAY "Parcours BFS :"

  PERFORM UNTIL WS-FRONT > WS-REAR
    *> Defiler
    MOVE WS-Q(WS-FRONT) TO WS-CURRENT
    ADD 1 TO WS-FRONT

    DISPLAY "Sommet : " WS-CURRENT

    *> Examiner tous les voisins du sommet courant
    PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-N
      IF WS-COL(WS-CURRENT, WS-I) = 1 AND WS-VIS(WS-I) = 0
```

```
            MOVE 1 TO WS-VIS(WS-I)
            ADD 1 TO WS-REAR
            MOVE WS-I TO WS-Q(WS-REAR)
          END-IF
        END-PERFORM
      END-PERFORM

      STOP RUN.
```

---

14. DFS très simple avec pile manuelle

Même idée, mais en profondeur.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. DFS-GRAPHE.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 WS-N              PIC 9 VALUE 4.
01 WS-I             PIC 9.
01 WS-CURRENT        PIC 9.

*> Pile
01 WS-TOP            PIC 9 VALUE 1.
01 WS-STACK.
  05 WS-S           PIC 9 OCCURS 10 TIMES VALUE 0.

01 WS-VISITED.
  05 WS-VIS         PIC 9 OCCURS 4 TIMES VALUE 0.

01 WS-GRAPH.
  05 WS-ROW OCCURS 4 TIMES.
    10 WS-COL       PIC 9 OCCURS 4 TIMES VALUE 0.

PROCEDURE DIVISION.
  *> Initialisation du graphe
  MOVE 1 TO WS-COL(1,2)
  MOVE 1 TO WS-COL(1,3)
  MOVE 1 TO WS-COL(2,4)
  MOVE 1 TO WS-COL(3,4)

  *> Push du sommet 1
  MOVE 1 TO WS-S(WS-TOP)
```

```
  DISPLAY "Parcours DFS :"

  PERFORM UNTIL WS-TOP = 0
     *> Pop
     MOVE WS-S(WS-TOP) TO WS-CURRENT
     SUBTRACT 1 FROM WS-TOP

     IF WS-VIS(WS-CURRENT) = 0
        MOVE 1 TO WS-VIS(WS-CURRENT)
        DISPLAY "Sommet : " WS-CURRENT

        *> On empile les voisins
        PERFORM VARYING WS-I FROM WS-N BY -1 UNTIL WS-I < 1
           IF WS-COL(WS-CURRENT, WS-I) = 1
              AND WS-VIS(WS-I) = 0
               ADD 1 TO WS-TOP
               MOVE WS-I TO WS-S(WS-TOP)
           END-IF
        END-PERFORM
     END-IF
  END-PERFORM

  STOP RUN.
```

---

15. Exemple très demandé en contexte COBOL : lire un fichier et compter les lignes

Ça, c’est typiquement plus réaliste en test COBOL qu’un graphe.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. COMPTER-LIGNES.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
  SELECT FICHIER-IN ASSIGN TO "input.txt"
     ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
FILE SECTION.
FD FICHIER-IN.
01 ENR-IN            PIC X(80).
```

```
WORKING-STORAGE SECTION.
01 WS-FIN            PIC X VALUE "N".
  88 FIN-FICHIER      VALUE "O".
01 WS-NB-LIGNES       PIC 9(5) VALUE 0.

PROCEDURE DIVISION.
   OPEN INPUT FICHIER-IN

   PERFORM UNTIL FIN-FICHIER
      READ FICHIER-IN
         AT END
            MOVE "O" TO WS-FIN
         NOT AT END
            ADD 1 TO WS-NB-LIGNES
      END-READ
   END-PERFORM

   CLOSE FICHIER-IN

   DISPLAY "Nombre de lignes : " WS-NB-LIGNES
   STOP RUN.
```

---

16. Lire un fichier et compter selon une condition

Exemple : compter le nombre de lignes contenant `OK`.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. COMPTER-STATUT-OK.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
   SELECT FICHIER-IN ASSIGN TO "statuts.txt"
      ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
FILE SECTION.
FD FICHIER-IN.
01 ENR-IN            PIC X(80).

WORKING-STORAGE SECTION.
01 WS-FIN            PIC X VALUE "N".
```

```
  88 FIN-FICHIER      VALUE "O".
01 WS-NB-OK           PIC 9(5) VALUE 0.

PROCEDURE DIVISION.
   OPEN INPUT FICHIER-IN

   PERFORM UNTIL FIN-FICHIER
      READ FICHIER-IN
         AT END
            MOVE "O" TO WS-FIN
         NOT AT END
            *> On teste les 2 premiers caracteres
            IF ENR-IN(1:2) = "OK"
               ADD 1 TO WS-NB-OK
            END-IF
      END-READ
   END-PERFORM

   CLOSE FICHIER-IN

   DISPLAY "Nombre de lignes OK : " WS-NB-OK
   STOP RUN.
```

Les questions les plus probables à réviser d’abord

Si tu veux bosser les classiques, révise surtout ceux-là :

1. **Bubble sort**
2. **Recherche linéaire**
3. **Recherche binaire**
4. **Max / min / somme / moyenne**
5. **Factorielle**
6. **Fibonacci**
7. **Palindrome / inversion de chaîne**
8. **Nombre premier**

9. **Fusion de tableaux**
10. **Lecture de fichier + comptage**
11. **BFS / DFS** juste pour culture algo

Petit conseil pour les examens COBOL

En pratique, on te demandera souvent plus :

- **des tableaux**
- **des chaînes**
- **des fichiers**
- **des conditions métier**
  que des graphes compliqués.

Donc si tu veux être efficace :

- apprends très bien `PERFORM`,
- `OCCURS`,
- `IF`,
- `EVALUATE`,
- `READ`,
- `WRITE`,
- `MOVE`,
- `COMPUTE`.

Parfait — voici une **fiche de révision : 20 exercices COBOL classiques avec correction**, en mode débutant, avec **commentaires**.

J'ai mis les exercices dans un ordre progressif :

- très simples,
- tableaux,
- chaînes,
- fichiers,
- puis un peu d'algo.

Les exemples sont écrits dans un style **lisible pour apprendre**.

Fiche — 20 exercices COBOL classiques avec correction

Exercice 1 — Afficher un message

Énoncé

Écrire un programme qui affiche `Bonjour COBOL`.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO1-BONJOUR.

PROCEDURE DIVISION.
   *> Affiche un texte a l'ecran
   DISPLAY "Bonjour COBOL"

   STOP RUN.
```

Exercice 2 — Lire un nom et l'afficher

Énoncé

Demander le nom de l'utilisateur puis afficher `Bonjour <nom>`.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO2-NOM.

DATA DIVISION.
```

```
WORKING-STORAGE SECTION.
01 WS-NOM          PIC X(20).

PROCEDURE DIVISION.
   DISPLAY "Entrez votre nom : "
   ACCEPT WS-NOM

   DISPLAY "Bonjour " WS-NOM

   STOP RUN.
```

Exercice 3 — Addition de deux nombres

Énoncé

Lire deux nombres entiers et afficher leur somme.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO3-ADDITION.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-A            PIC 9(3).
01 WS-B            PIC 9(3).
01 WS-SOMME        PIC 9(4).

PROCEDURE DIVISION.
   DISPLAY "Entrez le premier nombre : "
   ACCEPT WS-A

   DISPLAY "Entrez le second nombre : "
   ACCEPT WS-B

   *> Calcule la somme
   COMPUTE WS-SOMME = WS-A + WS-B

   DISPLAY "Somme = " WS-SOMME

   STOP RUN.
```

Exercice 4 — Dire si une personne est majeure

Énoncé

Lire un âge et afficher si la personne est majeure ou mineure.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO4-MAJEUR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-AGE          PIC 9(3).

PROCEDURE DIVISION.
   DISPLAY "Entrez l'age : "
   ACCEPT WS-AGE

   IF WS-AGE >= 18
      DISPLAY "Majeur"
   ELSE
      DISPLAY "Mineur"
   END-IF

   STOP RUN.
```

Exercice 5 — Échanger deux variables

Énoncé

Échanger le contenu de deux variables numériques.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO5-ECHANGE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-A            PIC 9(3) VALUE 10.
01 WS-B            PIC 9(3) VALUE 25.
01 WS-TMP          PIC 9(3).

PROCEDURE DIVISION.
   DISPLAY "Avant echange : A=" WS-A " B=" WS-B

   *> On stocke A dans une variable temporaire
```

```
   MOVE WS-A TO WS-TMP
   MOVE WS-B TO WS-A
   MOVE WS-TMP TO WS-B

   DISPLAY "Apres echange : A=" WS-A " B=" WS-B

   STOP RUN.
```

Exercice 6 — Afficher les nombres de 1 à 10

Énoncé

Afficher les nombres de 1 à 10 avec une boucle.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO6-BOUCLE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I          PIC 9(2) VALUE 1.

PROCEDURE DIVISION.
   PERFORM UNTIL WS-I > 10
      DISPLAY WS-I
      ADD 1 TO WS-I
   END-PERFORM

   STOP RUN.
```

Exercice 7 — Calculer la somme de 1 à N

Énoncé

Lire un nombre `N` et calculer `1 + 2 + ... + N`.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO7-SOMME-N.

DATA DIVISION.
```

```
WORKING-STORAGE SECTION.
01 WS-N            PIC 9(3).
01 WS-I            PIC 9(3).
01 WS-SOMME        PIC 9(5) VALUE 0.

PROCEDURE DIVISION.
   DISPLAY "Entrez N : "
   ACCEPT WS-N

   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-N
      ADD WS-I TO WS-SOMME
   END-PERFORM

   DISPLAY "Somme = " WS-SOMME

   STOP RUN.
```

Exercice 8 — Calculer une factorielle

Énoncé

Calculer la factorielle d'un nombre N.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO8-FACTORIELLE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-N            PIC 9(2).
01 WS-I            PIC 9(2).
01 WS-FACT         PIC 9(9) VALUE 1.

PROCEDURE DIVISION.
   DISPLAY "Entrez N : "
   ACCEPT WS-N

   IF WS-N = 0
      MOVE 1 TO WS-FACT
   ELSE
      PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-N
         MULTIPLY WS-I BY WS-FACT
      END-PERFORM
   END-IF
```

```
   DISPLAY "Factorielle = " WS-FACT

   STOP RUN.
```

Exercice 9 — Afficher la table de multiplication d’un nombre

Énoncé

Lire un nombre et afficher sa table de multiplication de 1 à 10.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO9-TABLE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-N            PIC 9(3).
01 WS-I            PIC 9(2).
01 WS-RES          PIC 9(5).

PROCEDURE DIVISION.
   DISPLAY "Entrez un nombre : "
   ACCEPT WS-N

   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 10
      COMPUTE WS-RES = WS-N * WS-I
      DISPLAY WS-N " x " WS-I " = " WS-RES
   END-PERFORM

   STOP RUN.
```

Exercice 10 — Trouver le maximum de trois nombres

Énoncé

Lire trois nombres et afficher le plus grand.

Correction

```
IDENTIFICATION DIVISION.
```

```
PROGRAM-ID. EXO10-MAX3.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-A          PIC 9(3).
01 WS-B          PIC 9(3).
01 WS-C          PIC 9(3).
01 WS-MAX         PIC 9(3).

PROCEDURE DIVISION.
   DISPLAY "Nombre 1 : "
   ACCEPT WS-A

   DISPLAY "Nombre 2 : "
   ACCEPT WS-B

   DISPLAY "Nombre 3 : "
   ACCEPT WS-C

   *> On suppose d'abord que A est le max
   MOVE WS-A TO WS-MAX

   IF WS-B > WS-MAX
      MOVE WS-B TO WS-MAX
   END-IF

   IF WS-C > WS-MAX
      MOVE WS-C TO WS-MAX
   END-IF

   DISPLAY "Maximum = " WS-MAX

   STOP RUN.
```

Exercice 11 — Compter les nombres pairs entre 1 et N

Énoncé

Lire un nombre N et compter combien de nombres pairs il y a entre 1 et N.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO11-PAIRS.
```

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-N            PIC 9(3).
01 WS-I            PIC 9(3).
01 WS-RESTE        PIC 9.
01 WS-CPT          PIC 9(3) VALUE 0.

PROCEDURE DIVISION.
   DISPLAY "Entrez N : "
   ACCEPT WS-N

   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-N
      DIVIDE WS-I BY 2 GIVING WS-RESTE REMAINDER WS-RESTE
      IF WS-RESTE = 0
         ADD 1 TO WS-CPT
      END-IF
   END-PERFORM

   DISPLAY "Nombre de pairs = " WS-CPT

   STOP RUN.
```

Exercice 12 — Recherche linéaire dans un tableau

Énoncé

Chercher une valeur dans un tableau.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO12-RECH-LIN.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I            PIC 9(2).
01 WS-CIBLE        PIC 9(3) VALUE 27.
01 WS-TROUVE       PIC X VALUE "N".
  88 TROUVE       VALUE "O".
01 WS-POS          PIC 9(2) VALUE 0.

01 WS-TAB.
  05 WS-VAL       PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
```

```
*> Initialisation du tableau
MOVE 10 TO WS-VAL(1)
MOVE 27 TO WS-VAL(2)
MOVE 33 TO WS-VAL(3)
MOVE 45 TO WS-VAL(4)
MOVE 50 TO WS-VAL(5)

PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5 OR TROUVE
   IF WS-VAL(WS-I) = WS-CIBLE
      MOVE "O" TO WS-TROUVE
      MOVE WS-I TO WS-POS
   END-IF
END-PERFORM

IF TROUVE
   DISPLAY "Trouve a la position " WS-POS
ELSE
   DISPLAY "Valeur non trouvee"
END-IF

STOP RUN.
```

Exercice 13 — Trouver min, max, somme, moyenne d’un tableau

Énoncé

À partir d’un tableau de 5 nombres, calculer le minimum, le maximum, la somme et la moyenne.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO13-STATS.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I          PIC 9(2).
01 WS-SOMME       PIC 9(5) VALUE 0.
01 WS-MAX         PIC 9(3).
01 WS-MIN         PIC 9(3).
01 WS-MOY         PIC 9(3)V99.

01 WS-TAB.
  05 WS-VAL       PIC 9(3) OCCURS 5 TIMES.
```

```
PROCEDURE DIVISION.
   MOVE 12 TO WS-VAL(1)
   MOVE 35 TO WS-VAL(2)
   MOVE 08 TO WS-VAL(3)
   MOVE 47 TO WS-VAL(4)
   MOVE 19 TO WS-VAL(5)

   *> Initialisation avec la 1ere valeur
   MOVE WS-VAL(1) TO WS-MAX
   MOVE WS-VAL(1) TO WS-MIN

   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
      ADD WS-VAL(WS-I) TO WS-SOMME

      IF WS-VAL(WS-I) > WS-MAX
         MOVE WS-VAL(WS-I) TO WS-MAX
      END-IF

      IF WS-VAL(WS-I) < WS-MIN
         MOVE WS-VAL(WS-I) TO WS-MIN
      END-IF
   END-PERFORM

   COMPUTE WS-MOY = WS-SOMME / 5

   DISPLAY "Somme   = " WS-SOMME
   DISPLAY "Min     = " WS-MIN
   DISPLAY "Max     = " WS-MAX
   DISPLAY "Moyenne = " WS-MOY

   STOP RUN.
```

Exercice 14 — Bubble sort

Énoncé

Trier un tableau par ordre croissant avec le tri à bulles.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO14-BUBBLE-SORT.

DATA DIVISION.
```

```
WORKING-STORAGE SECTION.
01 WS-I            PIC 9(2).
01 WS-J            PIC 9(2).
01 WS-TMP          PIC 9(3).

01 WS-TAB.
  05 WS-VAL        PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   MOVE 45 TO WS-VAL(1)
   MOVE 12 TO WS-VAL(2)
   MOVE 89 TO WS-VAL(3)
   MOVE 03 TO WS-VAL(4)
   MOVE 27 TO WS-VAL(5)

   DISPLAY "Avant tri :"
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
      DISPLAY WS-VAL(WS-I)
   END-PERFORM

   *> Bubble sort
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 4
      PERFORM VARYING WS-J FROM 1 BY 1 UNTIL WS-J > 5 - WS-I
         IF WS-VAL(WS-J) > WS-VAL(WS-J + 1)
            MOVE WS-VAL(WS-J)     TO WS-TMP
            MOVE WS-VAL(WS-J + 1) TO WS-VAL(WS-J)
            MOVE WS-TMP           TO WS-VAL(WS-J + 1)
         END-IF
      END-PERFORM
   END-PERFORM

   DISPLAY "Apres tri :"
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > 5
      DISPLAY WS-VAL(WS-I)
   END-PERFORM

   STOP RUN.
```

Exercice 15 — Vérifier si un mot est un palindrome

Énoncé

Vérifier si un mot se lit de la même façon dans les deux sens.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO15-PALINDROME.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTE        PIC X(20) VALUE "RADAR".
01 WS-LEN          PIC 9(2) VALUE 5.
01 WS-I            PIC 9(2).
01 WS-OK           PIC X VALUE "O".
  88 EST-PAL       VALUE "O".
  88 PAS-PAL       VALUE "N".

PROCEDURE DIVISION.
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-LEN / 2 OR PAS-PAL
      IF WS-TEXTE(WS-I:1) NOT = WS-TEXTE(WS-LEN - WS-I + 1:1)
         MOVE "N" TO WS-OK
      END-IF
   END-PERFORM

   IF EST-PAL
      DISPLAY "Palindrome"
   ELSE
      DISPLAY "Pas palindrome"
   END-IF

   STOP RUN.
```

Exercice 16 — Compter les voyelles dans une chaîne

Énoncé

Compter le nombre de voyelles d’un texte.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO16-VOYELLES.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTE        PIC X(20) VALUE "BONJOUR COBOL".
01 WS-I            PIC 9(2).
01 WS-CAR          PIC X.
01 WS-NB           PIC 9(2) VALUE 0.
01 WS-LEN          PIC 9(2) VALUE 13.
```

```
PROCEDURE DIVISION.
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-LEN
      MOVE WS-TEXTE(WS-I:1) TO WS-CAR

      EVALUATE WS-CAR
         WHEN "A"
         WHEN "E"
         WHEN "I"
         WHEN "O"
         WHEN "U"
         WHEN "Y"
            ADD 1 TO WS-NB
         WHEN OTHER
            CONTINUE
      END-EVALUATE
   END-PERFORM

   DISPLAY "Nombre de voyelles = " WS-NB

   STOP RUN.
```

Exercice 17 — Inverser une chaîne

Énoncé

Créer une deuxième chaîne contenant le texte à l’envers.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO17-INVERSE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-TEXTE        PIC X(20) VALUE "COBOL".
01 WS-INV          PIC X(20) VALUE SPACES.
01 WS-LEN          PIC 9(2) VALUE 5.
01 WS-I            PIC 9(2).

PROCEDURE DIVISION.
   PERFORM VARYING WS-I FROM 1 BY 1 UNTIL WS-I > WS-LEN
      MOVE WS-TEXTE(WS-LEN - WS-I + 1:1) TO WS-INV(WS-I:1)
   END-PERFORM
```

```
   DISPLAY "Texte   = " WS-TEXTE
   DISPLAY "Inverse = " WS-INV

   STOP RUN.
```

Exercice 18 — Recherche binaire

Énoncé

Chercher une valeur dans un tableau déjà trié avec la recherche binaire.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO18-RECH-BINAIRE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-LOW          PIC 9(2) VALUE 1.
01 WS-HIGH         PIC 9(2) VALUE 5.
01 WS-MID          PIC 9(2).
01 WS-CIBLE        PIC 9(3) VALUE 33.
01 WS-POS          PIC 9(2) VALUE 0.
01 WS-OK           PIC X VALUE "N".
  88 TROUVE        VALUE "O".

01 WS-TAB.
  05 WS-VAL        PIC 9(3) OCCURS 5 TIMES.

PROCEDURE DIVISION.
   MOVE 10 TO WS-VAL(1)
   MOVE 20 TO WS-VAL(2)
   MOVE 33 TO WS-VAL(3)
   MOVE 40 TO WS-VAL(4)
   MOVE 55 TO WS-VAL(5)

   PERFORM UNTIL WS-LOW > WS-HIGH OR TROUVE
      COMPUTE WS-MID = (WS-LOW + WS-HIGH) / 2

      IF WS-VAL(WS-MID) = WS-CIBLE
         MOVE "O" TO WS-OK
         MOVE WS-MID TO WS-POS
      ELSE
         IF WS-CIBLE < WS-VAL(WS-MID)
            SUBTRACT 1 FROM WS-MID GIVING WS-HIGH
```

```
        ELSE
           ADD 1 TO WS-MID GIVING WS-LOW
        END-IF
     END-IF
  END-PERFORM

  IF TROUVE
     DISPLAY "Trouve a la position " WS-POS
  ELSE
     DISPLAY "Non trouve"
  END-IF

  STOP RUN.
```

Exercice 19 — Lire un fichier et compter les lignes

Énoncé

Lire un fichier texte et compter le nombre de lignes.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO19-COMPTE-LIGNES.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
   SELECT FICHIER-IN ASSIGN TO "input.txt"
      ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
FILE SECTION.
FD FICHIER-IN.
01 ENR-IN          PIC X(80).

WORKING-STORAGE SECTION.
01 WS-FIN          PIC X VALUE "N".
  88 FIN-FICHIER   VALUE "O".
01 WS-NB-LIGNES     PIC 9(5) VALUE 0.

PROCEDURE DIVISION.
   OPEN INPUT FICHIER-IN

   PERFORM UNTIL FIN-FICHIER
```

```
      READ FICHIER-IN
         AT END
            MOVE "O" TO WS-FIN
         NOT AT END
            ADD 1 TO WS-NB-LIGNES
      END-READ
   END-PERFORM

   CLOSE FICHIER-IN

   DISPLAY "Nombre de lignes = " WS-NB-LIGNES

   STOP RUN.
```

Exercice 20 — Lire un fichier et filtrer les lignes commençant par OK

Énoncé

Lire un fichier texte et compter combien de lignes commencent par `OK`.

Correction

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EXO20-FILTRE-OK.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
   SELECT FICHIER-IN ASSIGN TO "statuts.txt"
      ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
FILE SECTION.
FD FICHIER-IN.
01 ENR-IN          PIC X(80).

WORKING-STORAGE SECTION.
01 WS-FIN          PIC X VALUE "N".
  88 FIN-FICHIER   VALUE "O".
01 WS-NB-OK        PIC 9(5) VALUE 0.

PROCEDURE DIVISION.
   OPEN INPUT FICHIER-IN
```

```
   PERFORM UNTIL FIN-FICHIER
      READ FICHIER-IN
         AT END
            MOVE "O" TO WS-FIN
         NOT AT END
            *> On teste les 2 premiers caracteres
            IF ENR-IN(1:2) = "OK"
               ADD 1 TO WS-NB-OK
            END-IF
      END-READ
   END-PERFORM

   CLOSE FICHIER-IN

   DISPLAY "Nombre de lignes OK = " WS-NB-OK

   STOP RUN.
```

Bonus — Deux exercices “algo” qu’on peut aussi te demander

Bonus A — Fibonacci

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BONUS-FIBO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-N          PIC 9(2) VALUE 10.
01 WS-I          PIC 9(2).
01 WS-A          PIC 9(5) VALUE 0.
01 WS-B          PIC 9(5) VALUE 1.
01 WS-C          PIC 9(5).

PROCEDURE DIVISION.
   DISPLAY WS-A
   DISPLAY WS-B

   PERFORM VARYING WS-I FROM 3 BY 1 UNTIL WS-I > WS-N
      COMPUTE WS-C = WS-A + WS-B
      DISPLAY WS-C
      MOVE WS-B TO WS-A
      MOVE WS-C TO WS-B
```

```
   END-PERFORM

   STOP RUN.
```

Bonus B — Vérifier si un nombre est premier

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BONUS-PREMIER.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-N            PIC 9(3) VALUE 29.
01 WS-I            PIC 9(3).
01 WS-RESTE        PIC 9(3).
01 WS-OK           PIC X VALUE "O".
  88 EST-PREMIER   VALUE "O".
  88 PAS-PREMIER   VALUE "N".

PROCEDURE DIVISION.
   IF WS-N < 2
      MOVE "N" TO WS-OK
   ELSE
      PERFORM VARYING WS-I FROM 2 BY 1 UNTIL WS-I >= WS-N OR PAS-PREMIER
         DIVIDE WS-N BY WS-I GIVING WS-RESTE REMAINDER WS-RESTE
         IF WS-RESTE = 0
            MOVE "N" TO WS-OK
         END-IF
      END-PERFORM
   END-IF

   IF EST-PREMIER
      DISPLAY WS-N " est premier"
   ELSE
      DISPLAY WS-N " n'est pas premier"
   END-IF

   STOP RUN.
```

Ce qu’il faut absolument savoir refaire sans aide

Pour un contrôle ou un entretien COBOL débutant, le plus important est de savoir refaire rapidement :

1. `DISPLAY`, `ACCEPT`, `MOVE`
2. `IF / ELSE / END-IF`
3. `PERFORM UNTIL`
4. `PERFORM VARYING`
5. `COMPUTE`
6. tableaux avec `OCCURS`
7. lecture de fichier avec `READ ... AT END`
8. tri simple et recherche simple
9. manipulation de chaînes avec `WS-TEXTE(pos:len)`
10. usage des niveaux `88`

Mini gabarit à apprendre par cœur

```
IDENTIFICATION DIVISION.
PROGRAM-ID. MONPROG.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I          PIC 9(2) VALUE 0.

PROCEDURE DIVISION.
   DISPLAY "Debut du programme"

   *> Traitement
   ADD 1 TO WS-I

   DISPLAY "Valeur = " WS-I
   STOP RUN.
```

Astuces d’examen

Les erreurs les plus fréquentes :

- oublier `END-IF`
- oublier `END-PERFORM`
- mettre une valeur texte dans une variable numérique
- oublier d’initialiser les variables

- mal dimensionner un `PIC`
- oublier `OPEN` avant `READ`
- oublier `CLOSE` après traitement fichier

Plan de révision conseillé

Révise dans cet ordre :

Jour 1

- exo 1 à 6

Jour 2

- exo 7 à 10

Jour 3

- exo 11 à 14

Jour 4

- exo 15 à 18

Jour 5

- exo 19, 20 + bonus

Je peux aussi te faire une **version encore plus utile en mode “PDF de révision propre”** ou une **version “questions d’examen + réponses courtes à apprendre”**.